System Mathematics

A two-joint arm (shoulder elevation, elbow flexion) is controlled by a PD controller whose output is augmented by a cerebellar feedforward term derived from an Echo State Network (ESN) forward model. The ESN is trained online with Recursive Least Squares (RLS).

---

1 Arm State

The arm has $n = 2$ degrees of freedom. At each simulation step t the observed state is:

Symbol	Dimension	Meaning
$\mathbf{q}(t)$	$\mathbb{R}^2$	joint angles (rad)
$\dot{\mathbf{q}}(t)$	$\mathbb{R}^2$	joint angular velocities (rad s^{-1})
$\ddot{\mathbf{q}}(t)$	$\mathbb{R}^2$	joint angular accelerations (rad s^{-2})
$\mathbf{q}^*$	$\mathbb{R}^2$	goal joint angles from IK (rad)

The simulation step size is $\Delta t = 0.005$ s.

---

2 PD Motor Controller

The baseline motor command is a proportional-derivative (PD) controller. A new torque command is computed every simulation step but **issued** (latched to the actuators) only every T_{PID} steps (zero-order hold between updates):

$$\boldsymbol{\tau}_{\text{PD}}(t) = K_p \odot (\mathbf{q}^* - \mathbf{q}(t)) - K_d \odot \dot{\mathbf{q}}(t)$$

where $\odot$ denotes element-wise multiplication with per-joint gain vectors.

Gain values

Joint	K_p (Nm rad^{-1})	K_d (Nm s rad^{-1})
`shoulder_elv`	160	45
`elbow_flexion`	90	22

The output is clamped:

$$\tau_{\mathrm{PD},i} \leftarrow \mathrm{clip}(\tau_{\mathrm{PD},i},\ -\tau_i^{\max},\ \tau_i^{\max})$$

with limits $\tau^{\max} = \{60, 30\}$ Nm.

Rate-limiting (zero-order hold)

The PD controller recomputes every tick but only writes a new command to the actuators every

$$N_{\mathrm{PID}} = \left\lfloor \frac{T_{\mathrm{PID}}}{\Delta t} \right\rfloor \text{ steps} \qquad (T_{\mathrm{PID}} = 30 \text{ ms})$$

Between updates the previous torque is held constant — the same held torque is delivered to the ESN as the efference copy.

---

3 Echo State Network — Cerebellar Forward Model

3.1 Architecture

The ESN is a recurrent neural network with a **fixed** random reservoir and a **trained linear readout**:

```
u(t) ──W_in──▶ r(t+1) ──W_out──▶ ŷ(t+1)
                ▲
               W_res (fixed)
```

3.2 Input Vector

All inputs are normalised to $[-1, 1]$ before entering the reservoir:

$$\mathbf{u}(t) = \begin{bmatrix} \mathbf{q}(t)/\pi \\ \dot{\mathbf{q}}(t)/15 \\ \boldsymbol{\tau}_{\mathrm{total}}(t)/\tau^{\max} \\ \mathbf{q}^*/\pi \end{bmatrix} \in \mathbb{R}^8$$

Block	Elements	Scale
$\mathbf{q}$	2	$\div \pi$ rad
$\dot{\mathbf{q}}$	2	$\div 15$ rad s^{-1}
$\boldsymbol{\tau}_{\text{total}}$	2	$\div \tau^{\max}$ Nm
$\mathbf{q}^*$	2	$\div \pi$ rad

$\boldsymbol{\tau}_{\text{total}} = \boldsymbol{\tau}_{\text{PD}} + \boldsymbol{\tau}_{\text{DCN}}$ is the **efference copy** — the full torque actually applied to the arm.

3.3 Reservoir Dynamics (leaky integrator)

$$\mathbf{r}(t+1) = (1-\alpha)\mathbf{r}(t) + \alpha \tanh(W_{\text{res}}\mathbf{r}(t) + W_{\text{in}}\mathbf{u}(t) + \mathbf{b})$$

Symbol	Value	Meaning
α	0.3	leak rate
$\rho(W_{\text{res}})$	0.95	spectral radius (echo-state property)
connectivity	10 %	fraction of non-zero entries in W_{res}
N	3500	reservoir nodes

W_{res}, W_{in}, and $\mathbf{b}$ are drawn randomly at construction and **never updated**.

3.4 Quadratic Augmentation

The readout uses a quadratic feature map to give the linear output access to pairwise neuron interactions:

$$\boldsymbol{\phi}(\mathbf{r}) = \begin{bmatrix} \mathbf{r} \\ \mathbf{r} \odot \mathbf{r} \\ 1 \end{bmatrix} \in \mathbb{R}^{2N+1}$$

3.5 Readout (Prediction)

$$\hat{\mathbf{y}}(t+1) = W_{\text{out}}\boldsymbol{\phi}(\mathbf{r}(t+1)) \in \mathbb{R}^4$$

The 4-element output encodes predicted **normalised** next state:

$$\hat{\mathbf{y}} = \begin{bmatrix} \hat{\mathbf{q}}(t+1)/\pi \\ \dot{\hat{\mathbf{q}}}(t+1)/15 \end{bmatrix}$$

Denormalised predictions used downstream:

$$\hat{\mathbf{q}}(t+1) = \hat{\mathbf{y}}_{0:2} \times \pi \qquad \dot{\hat{\mathbf{q}}}(t+1) = \hat{\mathbf{y}}_{2:4} \times 15$$

---

4 Online Learning — Recursive Least Squares (RLS)

W_{out} is updated online after every simulation step using the **Sherman–Morrison rank-1 RLS update** (equivalent to the climbing-fibre correction signal in the cerebellum).

4.1 Error Signal

$$\mathbf{e}(t) = \mathbf{y}^*(t) - \hat{\mathbf{y}}(t)$$

where $\mathbf{y}^*(t)$ is the normalised ground-truth next state observed after integration.

The Euclidean position error (in physical units):

$$\|\Delta q\| = \|\mathbf{e}_{0:2}\| \times \pi \quad (\text{rad})$$

4.2 Threshold Gate (Inferior Olive Model)

The RLS update fires **only** when the prediction error exceeds a threshold (analogous to the inferior olive → climbing-fibre pathway):

$$\text{update if } \|\Delta q\| > \theta_{\text{learn}} \qquad (\theta_{\text{learn}} = 0.05 \text{ rad} \approx 3^\circ)$$

Updates are additionally suppressed for $N_{\text{washout}} = 30$ steps after each new target is assigned (avoids corrupting P with the high-velocity transition spike).

4.3 RLS Weight Update

$$\mathbf{k}(t) = \frac{P(t-1)\boldsymbol{\phi}(t)}{\lambda + \boldsymbol{\phi}(t)^\top P(t-1)\boldsymbol{\phi}(t)}$$

$$W_{\text{out}}(t) \leftarrow W_{\text{out}}(t-1) + \mathbf{e}(t) \otimes \mathbf{k}(t)$$

$$P(t) \leftarrow \frac{1}{\lambda}\left[P(t-1) - \mathbf{k}(t)\boldsymbol{\phi}(t)^\top P(t-1)\right]$$

Symbol	Value	Meaning
λ	0.999	forgetting factor (1 = no forgetting)
δ	10^4	initial $P = \delta I$ (high $\rightarrow$ fast early learning)
$\boldsymbol{\phi}(t)$	$\in \mathbb{R}^{2N+1}$	augmented reservoir state at prediction time

---

5 Deep Cerebellar Nuclei (DCN) Feedforward Torque

The ESN's predicted next joint position is used to compute an anticipatory feedforward torque — a proxy for the Purkinje cell → DCN pathway:

$$\boldsymbol{\tau}_{\text{DCN}}(t) = K_{\text{DCN}} \odot (\mathbf{q}^* - \hat{\mathbf{q}}(t+1))$$

Joint	K_{DCN} (Nm rad^{-1})
`shoulder_elv`	15
`elbow_flexion`	10

Clamped to ± 75 Nm per joint.

5.1 Total Applied Torque

$$\boldsymbol{\tau}_{\text{total}}(t) = \boldsymbol{\tau}_{\text{PD}}(t) + \boldsymbol{\tau}_{\text{DCN}}(t)$$

This is both applied to the arm **and** fed back into the ESN as the efference copy at the next step.

---

6 Forward Kinematics (Ghost Arm)

To visualise the ESN prediction, predicted joint angles are passed through the OpenSim forward kinematics model:

$$(\mathbf{p}_{\text{shoulder}},\ \mathbf{p}_{\text{elbow}},\ \mathbf{p}_{\text{hand}}) = \mathrm{FK}(\hat{\mathbf{q}}(t+1)) \quad \in \mathbb{R}^3 \text{ (Ground frame, m)}$$

This is computed numerically via the Simbody engine on a separate model instance so the simulation state is never modified.

---

7 Target Sampling & Inverse Kinematics

Reach targets are sampled uniformly in Cartesian space within a reachable bounding box and mapped back to joint angles via a numerical Jacobian IK solver:

$$\mathbf{q}_{k+1} = \mathbf{q}_k + \alpha J(\mathbf{q}_k)^{+}(\mathbf{p}^* - \mathrm{FK}(\mathbf{q}_k))$$

where $J^{+} = J^{\top}(JJ^{\top})^{-1}$ is the Moore–Penrose pseudoinverse and $\alpha = 0.5$ is a step-size damping factor. Iteration stops when the hand-to-target Cartesian error falls below 0.5 mm.

A reach is declared complete when:

$$\|\mathbf{p}_{\text{hand}}(t) - \mathbf{p}^*\| < 0.09 \text{ m} \quad \text{and} \quad \|\dot{\mathbf{q}}(t)\|_\infty < 0.6 \text{ rad s}^{-1} \quad \text{and} \quad t - t_{\text{start}} > 0.12 \text{ s}$$

---

8 Convergence Detection (Diagnostic)

An exponential moving average of the position error is tracked:

$$\mu_q(t) = \beta\|\Delta q(t)\| + (1-\beta)\mu_q(t-1) \qquad \beta = 0.05$$

A convergence milestone is logged (but learning continues) when $\mu_q < 0.03$ rad for 400 consecutive steps.

---

9 Summary of Control Loop (Per Tick)

```
1.  Read q, qd, qdd from OpenSim state
2.  υ = normalise(q, qd, τ_total_prev, q*)          ← 8-element ESN input
3.  r ← leaky-integrator update(r, υ)               ← reservoir step
4.  ŷ = W_out · φ(r)                                ← predict q̂, q̂d (normalised)
5.  τ_PD  = Kp·(q*−q) − Kd·qd                      ← PD (issued every 30 ms)
6.  τ_DCN = K_DCN · (q* − q̂·π)                     ← cerebellar feedforward
7.  τ_total = τ_PD + τ_DCN                          ← apply to arm
8.  Integrate physics one step (Δt = 5 ms)
9.  Observe q', qd'  →  e = normalise(q',qd') − ŷ  ← prediction error
10. If ‖Δq‖ > θ_learn: RLS update W_out            ← online learning
```